



Università degli Studi di Verona
Corso di Progettazione di App React

**Dashboard React per il controllo remoto di sessioni di videoregistrazione su
NVIDIA Jetson Orin**

**Tommaso Capuzzi
Matricola: VR503278**

Indice

1	Introduzione	3
2	Obiettivi del progetto	3
3	Architettura del sistema	3
4	Descrizione delle API	4
5	Implementazione	5
5.1	Frontend	5
5.2	Backend	6
6	Gestione dei log	6
7	Sviluppi futuri	6
8	Conclusioni	7
9	Esempio di esecuzione del sistema	7

1 Introduzione

Questo progetto è stato sviluppato nell'ambito del corso *Progettazione di App React*. L'obiettivo è la realizzazione di una dashboard web che consenta il controllo remoto di sessioni di videoregistrazione utilizzate per il monitoraggio di pazienti affetti da malattia di Parkinson.

Il sistema permette ad un operatore di avviare e interrompere una pipeline di applicativi eseguiti su una scheda embedded NVIDIA Jetson Orin. Tale pipeline gestisce l'acquisizione video da due telecamere, la conversione dei dati video in formato raw e il salvataggio dei dati su storage.

La dashboard è stata sviluppata utilizzando React per il frontend e Node.js con Express per il backend.

2 Obiettivi del progetto

Gli obiettivi principali del progetto sono:

- fornire un'interfaccia web intuitiva per il controllo delle sessioni di registrazione
- consentire l'avvio e l'arresto della pipeline di acquisizione video
- monitorare lo stato dei diversi componenti del sistema
- visualizzare i log generati durante l'esecuzione della pipeline
- permettere il download dei log per analisi successive

3 Architettura del sistema

Il sistema sviluppato è composto da due componenti principali: un frontend realizzato con React e un backend sviluppato in Node.js con il framework Express.

La dashboard web rappresenta l'interfaccia utilizzata dall'operatore per controllare e monitorare le sessioni di videoregistrazione. Attraverso l'interfaccia grafica è possibile avviare o interrompere una sessione di registrazione e visualizzare lo stato dei diversi componenti della pipeline.

Il backend espone una serie di API REST che permettono al frontend di comunicare con il sistema. Tali API gestiscono lo stato della pipeline e simulano il comportamento della scheda NVIDIA Jetson Orin.

La pipeline di acquisizione video è composta da quattro componenti principali:

- acquisizione video dalla Camera 1
- acquisizione video dalla Camera 2
- conversione dei video nel formato raw
- salvataggio dei dati su storage

Nel progetto sviluppato queste operazioni sono simulate all'interno del backend per consentire lo sviluppo e il test della dashboard anche in assenza della scheda Jetson.

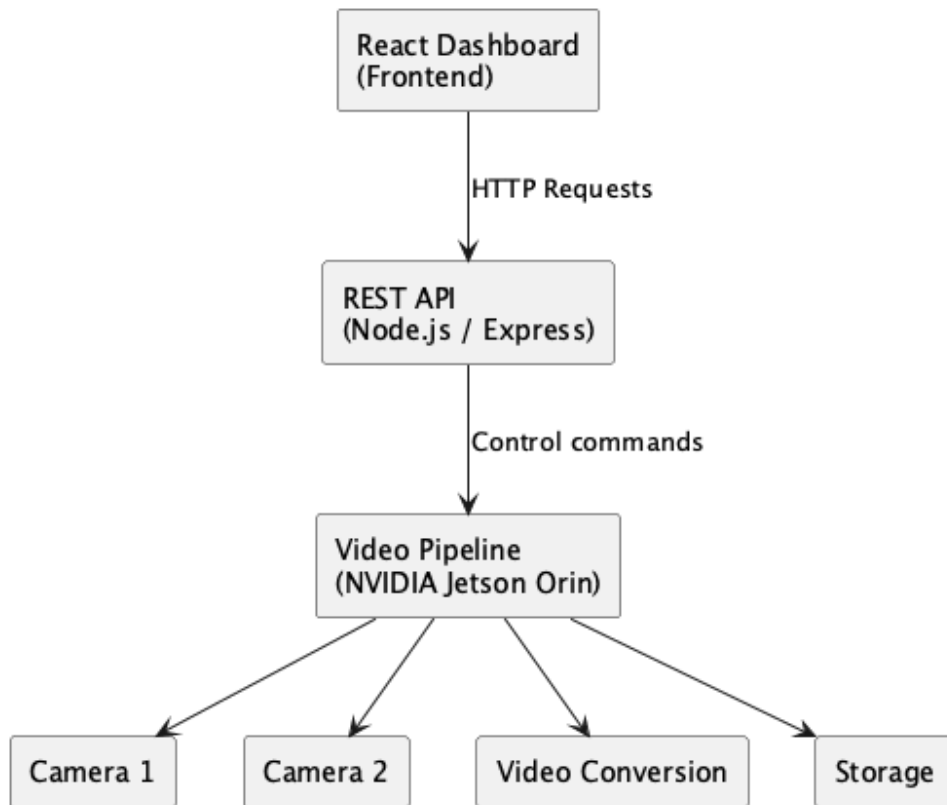


Figura 1: Architettura del sistema

4 Descrizione delle API

Il backend del sistema espone una serie di API REST che permettono alla dashboard React di comunicare con il server e controllare lo stato della pipeline di videoregistrazione.

API sono implementate utilizzando il framework Express in Node.js e utilizzano il protocollo HTTP per la comunicazione tra frontend e backend.

- **GET /api/session/status**

Questa API restituisce lo stato corrente della pipeline di registrazione. La risposta contiene informazioni sullo stato della sessione, il momento di avvio della registrazione e lo stato dei diversi componenti del sistema.

Esempio di risposta:

```
{
  "status": "RUNNING",
  "startedAt": "2026-03-03T17:30:36.436Z",
  "components": {
    "camera1": "ON",
    "camera2": "ON",
    "conversion": "ON",
    "storage": "ON"
  }
}
```

- **POST /api/session/start**

Questa API permette di avviare una nuova sessione di videoregistrazione. Quando viene chiamata, il backend aggiorna lo stato della pipeline e attiva i diversi componenti del sistema.

Funzionalità principali:

- inizializzazione della sessione di registrazione
- attivazione delle telecamere
- avvio della conversione dei dati video
- avvio del salvataggio dei dati su storage

- **POST /api/session/stop**

Questa API consente di interrompere la sessione di registrazione attualmente attiva.

Durante questa operazione il sistema esegue:

- arresto della conversione video
- interruzione della registrazione delle telecamere
- salvataggio finale dei dati
- aggiornamento dello stato della pipeline

- **GET /api/logs**

Questa API restituisce i log generati durante l'esecuzione della pipeline. I log contengono informazioni utili per monitorare il comportamento del sistema e diagnosticare eventuali problemi.

La dashboard React utilizza questa API per mostrare i log in tempo reale all'interno dell'interfaccia utente.

- **GET /api/logs/download**

Questa API permette di scaricare i log della sessione in formato file di testo. Questa funzionalità consente agli operatori di salvare i log per analisi successive o per attività di debugging.

5 Implementazione

5.1 Frontend

Il frontend è stato sviluppato utilizzando React con TypeScript e Vite.

La dashboard permette di:

- avviare una sessione di registrazione
- fermare una sessione di registrazione
- visualizzare lo stato della pipeline
- monitorare lo stato dei componenti del sistema
- visualizzare i log in tempo reale
- scaricare i log generati durante l'esecuzione

L'interfaccia è organizzata in diversi componenti React tra cui:

- **StatusCards:** visualizzazione dello stato dei componenti
- **LogPanel:** visualizzazione dei log della pipeline
- controlli per avvio e arresto delle sessioni

5.2 Backend

I

l backend è stato sviluppato utilizzando Node.js ed Express.

Esso espone diverse API REST che permettono di controllare lo stato della pipeline:

- GET /api/session/status
- POST /api/session/start
- POST /api/session/stop
- GET /api/logs
- GET /api/logs/download

Il backend mantiene uno stato interno della sessione e dei componenti della pipeline simulata.

6 Gestione dei log

Durante l'esecuzione della pipeline il sistema genera log che descrivono le diverse fasi del processo, tra cui:

- avvio della sessione
- attivazione delle telecamere
- conversione dei dati video
- salvataggio dei dati
- arresto della pipeline

I log vengono visualizzati in tempo reale nella dashboard e possono essere scaricati dall'operatore in formato testo.

7 Sviluppi futuri

Il progetto sviluppato rappresenta una prima implementazione di una dashboard per il controllo remoto di sessioni di videoregistrazione. In un contesto reale il sistema potrebbe essere esteso introducendo ulteriori funzionalità.

Tra i possibili sviluppi futuri si possono considerare:

- integrazione diretta con la scheda NVIDIA Jetson Orin per il controllo reale delle telecamere
- implementazione di un sistema di autenticazione per la gestione degli utenti
- visualizzazione dello streaming video in tempo reale all'interno della dashboard
- gestione di più sessioni di registrazione contemporaneamente

- integrazione con un database per la gestione e l'archiviazione dei dati raccolti

Questi sviluppi permetterebbero di trasformare il sistema in una piattaforma completa per la gestione di sessioni di videoregistrazione in ambito clinico e di ricerca.

8 Conclusioni

Il progetto ha portato alla realizzazione di una dashboard web che consente il controllo remoto di una pipeline di acquisizione video.

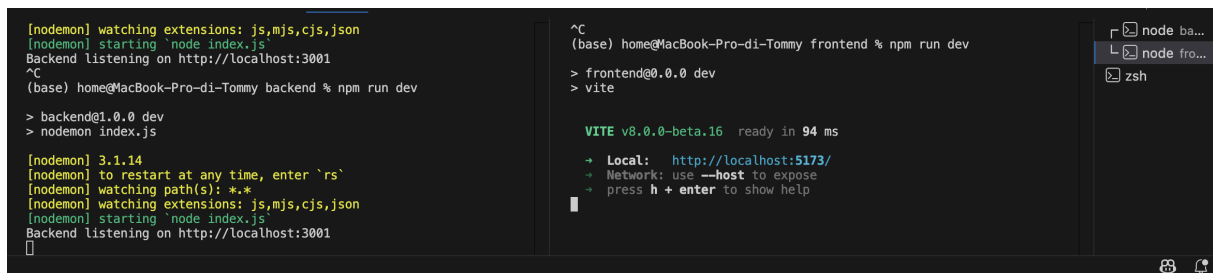
La struttura modulare del sistema permette di integrare facilmente la dashboard con una pipeline reale eseguita su una scheda NVIDIA Jetson Orin.

Possibili sviluppi futuri includono:

- integrazione con una pipeline reale su hardware Jetson
- supporto per configurazione avanzata delle sessioni
- miglioramento delle funzionalità di monitoraggio del sistema

9 Esempio di esecuzione del sistema

In questa sezione viene mostrato un esempio di esecuzione del progetto. Le immagini seguenti illustrano le principali fasi di utilizzo della dashboard, dall'avvio del sistema fino all'esecuzione della pipeline di acquisizione video.



```
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node index.js'
Backend listening on http://localhost:3001
^C
(base) home@MacBook-Pro-di-Tommy backend % npm run dev
> backend@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node index.js'
Backend listening on http://localhost:3001
^C
(base) home@MacBook-Pro-di-Tommy frontend % npm run dev
> frontend@0.0.0 dev
> vite

VITE v8.0.0-beta.16 ready in 94 ms
+ Local: http://localhost:5173/
+ Network: use --host to expose
+ press h + enter to show help
```

Figura 2: Esecuzione del progetto nel terminale. A sinistra viene avviato il backend Node.js con nodemon sulla porta 3001, mentre a destra viene avviato il frontend React tramite Vite sulla porta 5173.

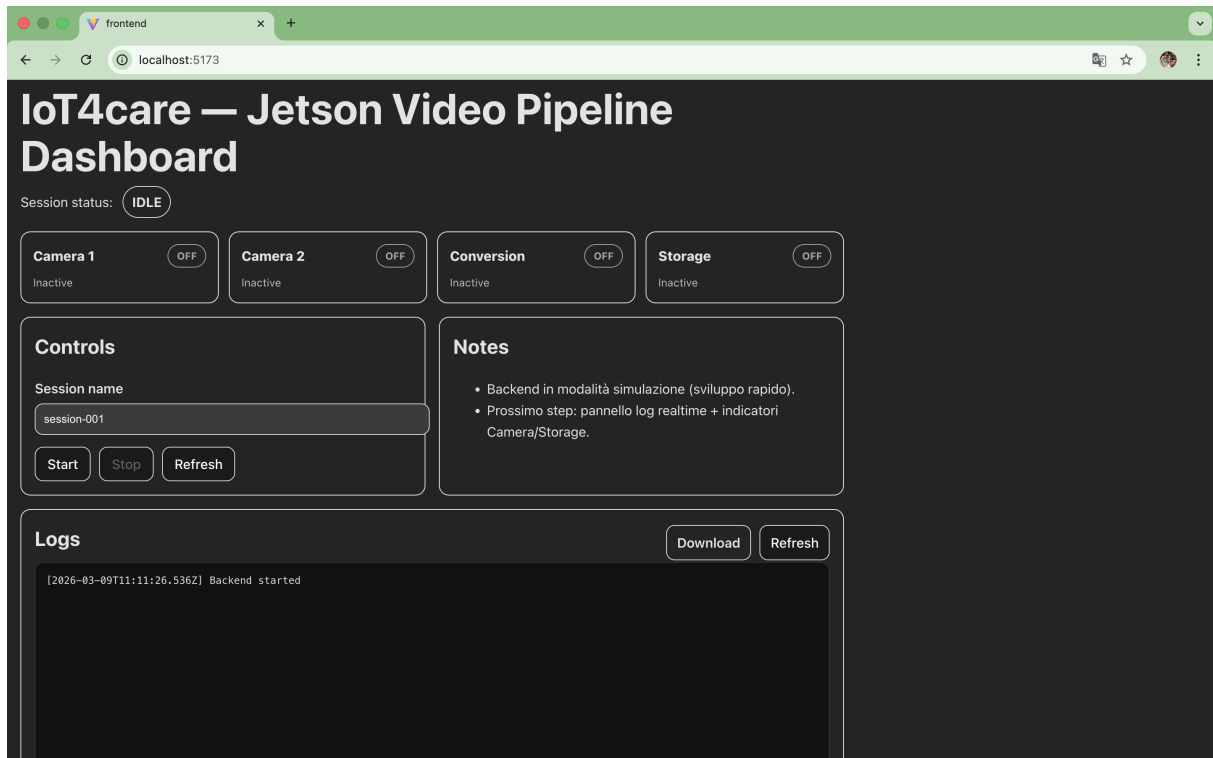


Figura 3: Dashboard React nello stato iniziale. Tutti i componenti della pipeline risultano inattivi e l'operatore può avviare una nuova sessione di registrazione tramite i controlli presenti nell'interfaccia.

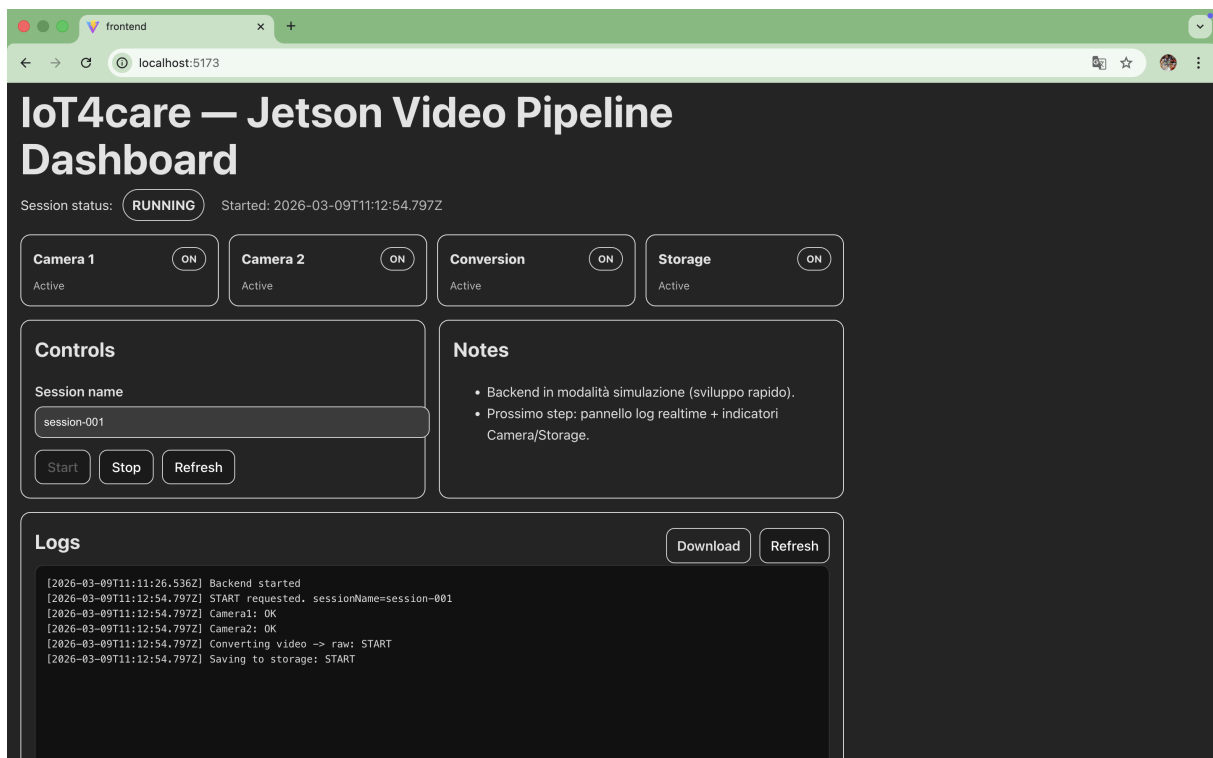


Figura 4: Dashboard durante l'esecuzione della pipeline. Dopo l'avvio della sessione di registrazione tutti i componenti risultano attivi e il pannello dei log mostra le operazioni eseguite dal sistema.

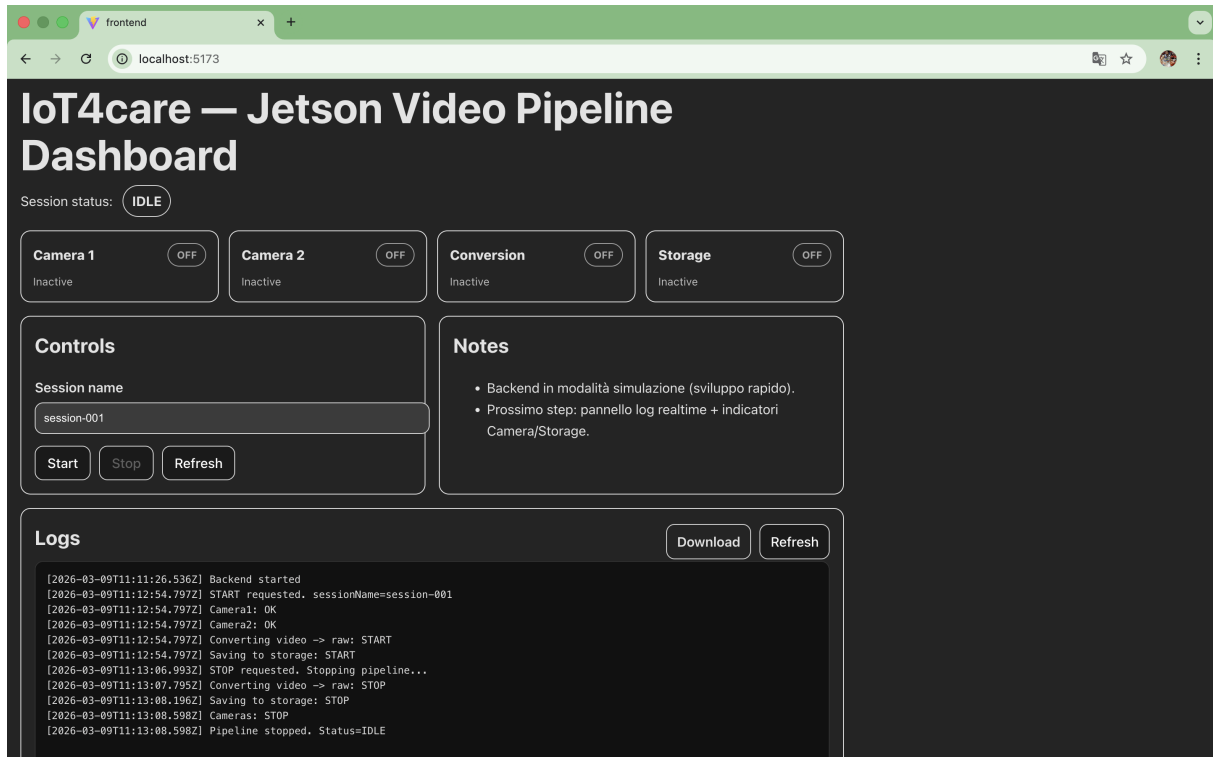


Figura 5: Dashboard dopo l'arresto della pipeline. Dopo aver premuto il pulsante Stop il sistema interrompe la registrazione, arresta i componenti della pipeline e i log mostrano le operazioni di terminazione della sessione.

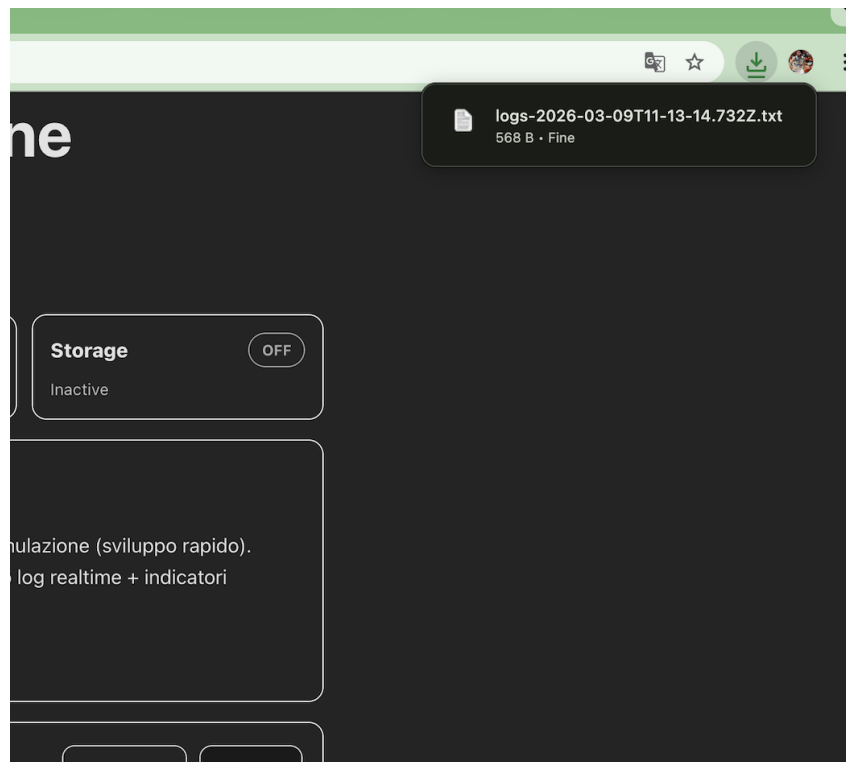
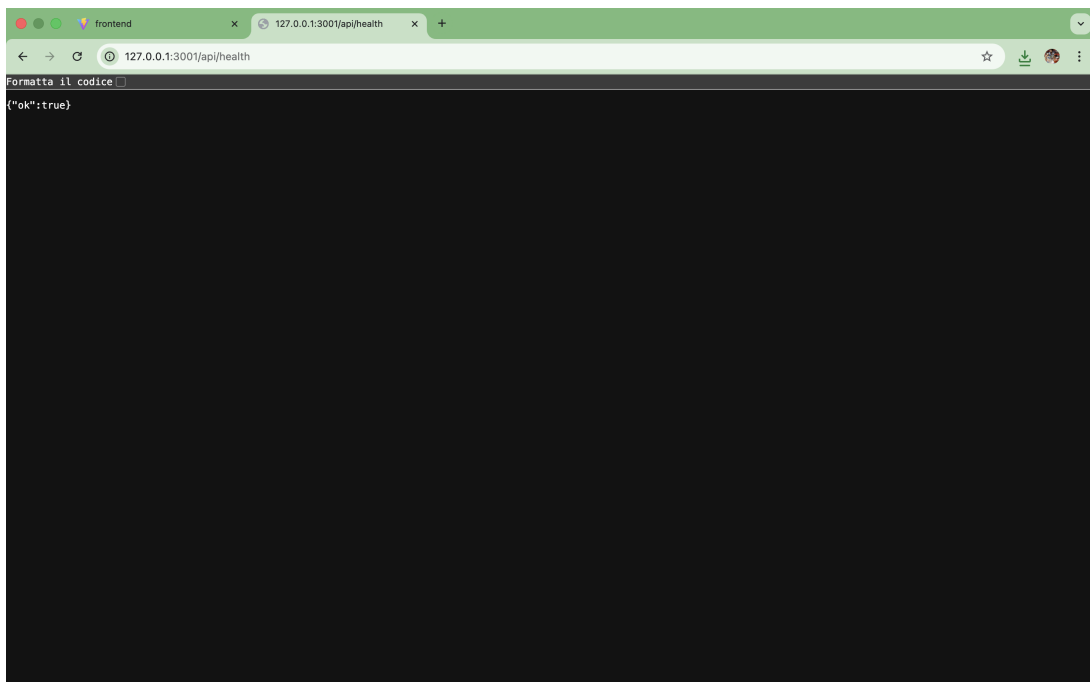


Figura 6: Download dei log della sessione tramite il pulsante Download presente nella dashboard. Il sistema genera un file di testo contenente i log prodotti durante l'esecuzione della pipeline.

```
[2026-03-09T11:11:20.530Z] Backend started
[2026-03-09T11:12:54.797Z] START requested. sessionName=session-001
[2026-03-09T11:12:54.797Z] Camera1: OK
[2026-03-09T11:12:54.797Z] Camera2: OK
[2026-03-09T11:12:54.797Z] Converting video -> raw: START
[2026-03-09T11:12:54.797Z] Saving to storage: START
[2026-03-09T11:13:06.263Z] STOP requested. Stopping pipeline...
[2026-03-09T11:13:07.795Z] Converting video -> raw: STOP
[2026-03-09T11:13:08.196Z] Saving to storage: STOP
[2026-03-09T11:13:08.598Z] Camera3: STOP
[2026-03-09T11:13:08.598Z] Pipeline stopped. Status=IDLE
```

Figura 7: Contenuto del file di log scaricato dalla dashboard. Il file contiene la sequenza delle operazioni eseguite dalla pipeline, tra cui l'avvio della sessione, l'attivazione delle telecamere, la conversione dei dati video e l'arresto del sistema.



The screenshot shows a web browser window with two tabs. The active tab is titled '127.0.0.1:3001/api/health'. The address bar shows the URL '127.0.0.1:3001/api/health'. Below the address bar, there is a text input field with the placeholder 'formatta il codice'. The main content area of the browser displays the JSON response: `{"ok":true}`.

Figura 8: Verifica dello stato del backend tramite l'endpoint `/api/health`. La risposta `{"ok": true}` indica che il server Node.js è correttamente avviato e operativo.

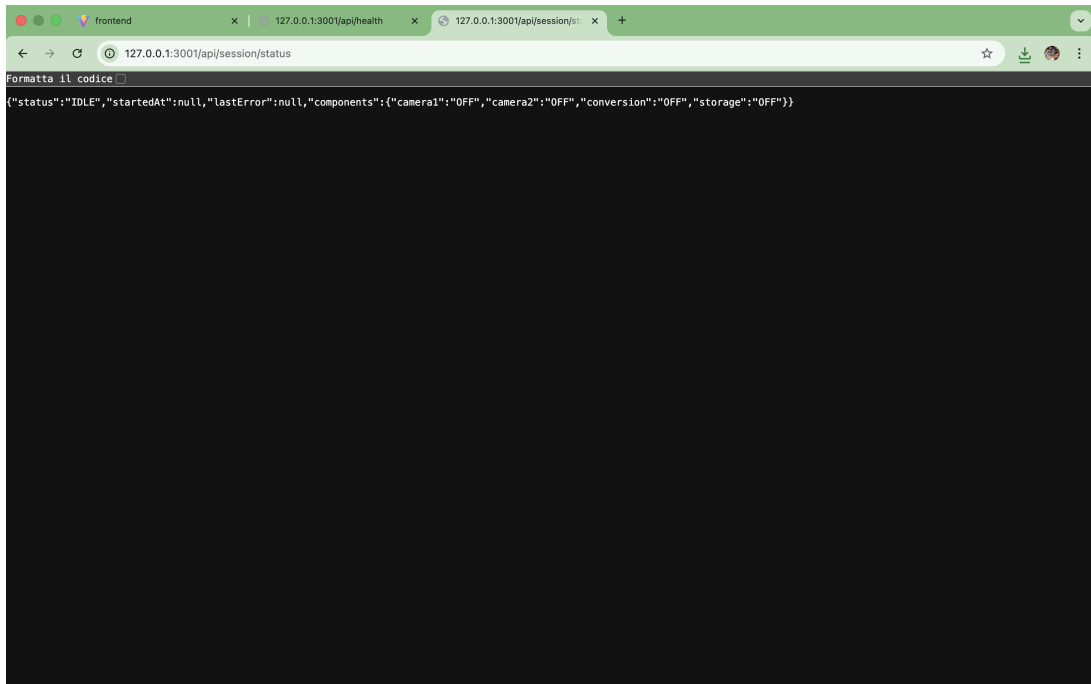


Figura 9: Risposta dell'endpoint `/api/session/status` quando la pipeline non è attiva. Tutti i componenti risultano nello stato `OFF`.

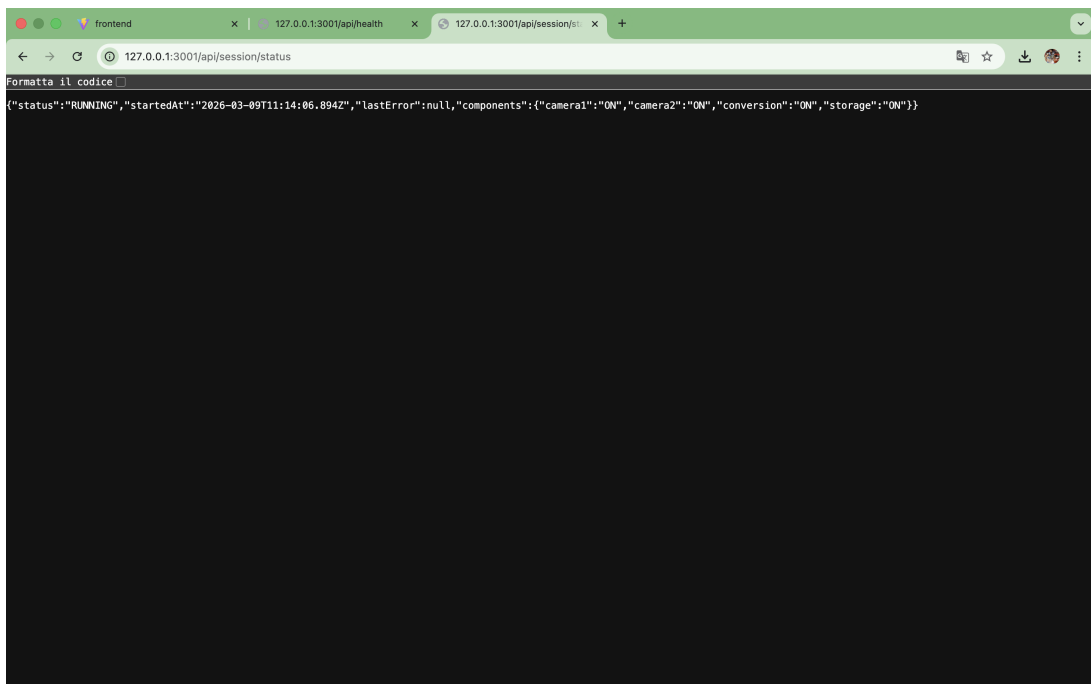


Figura 10: Risposta dell'endpoint `/api/session/status` durante l'esecuzione della pipeline. I componenti Camera, Conversion e Storage risultano attivi.